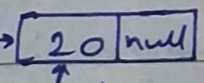
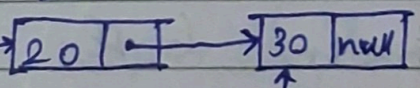
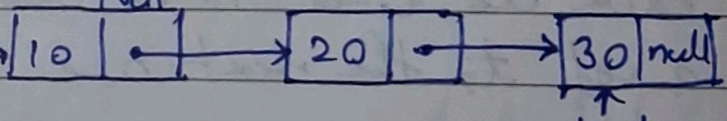
(*) Lab task: Singly Linked List.

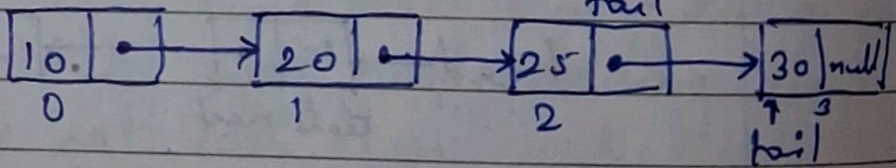
Method	Time	Space	Reason
middle()	O(n)	O(1)	slow/fast pointers
count(T, x)	O(n)	O(1)	visit every node
insertSorted(T, x)	O(n)	O(1)	find correct point

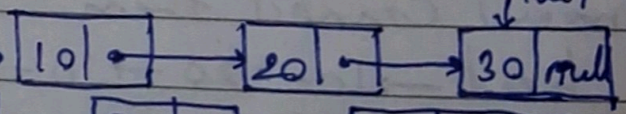
Dry Run:

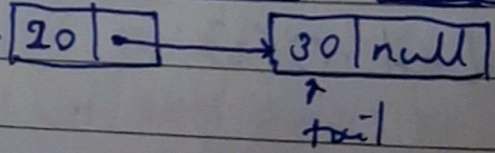
1. Initial values head = null, tail = null, size = 0.
2. addLast(20): 

, size = 1
3. addLast(30): 

, size = 2.
4. addFirst(10): 

, size 3.
5. add(2, 25): 

size = 4
6. indexOf(30) = 3
get(3) = 30, size = 5
7. remove(25): 

8. removeFirst(): 

9. addLast(50): head → 20 → 30 → 50 → null
tail

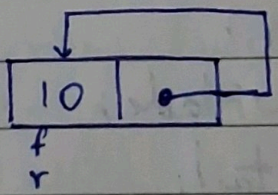
10. addLast(60): head → 20 → 30 → 50 → 60 → null
tail

11. reverse(): prev = null, cur = 20, tail = 20 (tail = head)
1. prev = 20, cur = 30 after reverse: 30 → 20 → null
2. prev = 30, cur = 50 after reverse: 50 → 30 → 20 → null
3. prev = 50, cur = 60 after reverse: 60 → 50 → 30 → 20 → null.
∴ head → 60 → 50 → 30 → 20 → null
tail

12. middle(): middle = 30
13. count(50): count = 1
14. insertSorted(35): head → 35 → 50 → 30 → 20 → null
It is not sorted because list was reversed already.

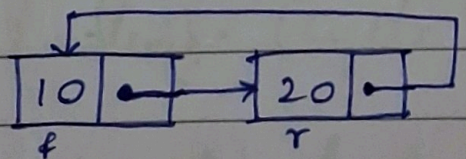
(*) Circular linked list: (enqueue)

(1) Queue is empty: rear = null
n = 10 (add), n.next = n;



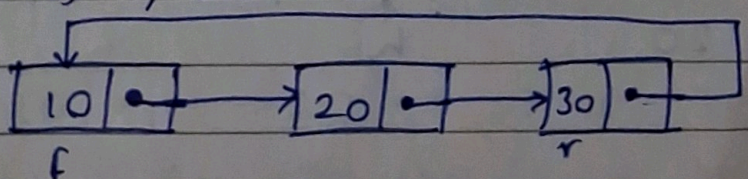
rear = 10
rear.next = 10

(2) rear = 10
n = 20 (add), rear.next = 20 = 10.next = 20



front = 10, rear = 20
rear.next = 10

(3) rear = 20
n = 30 (add/create), 20.next = 30

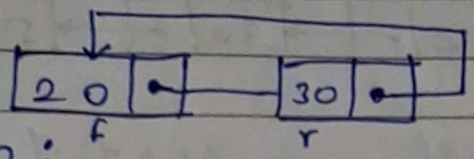


rear = 30
rear.next = 10

(dequeue)

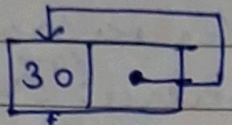
(1) rear = 30, front = 10

Remove 10 : rear.next = front.next;
front.next = 20, ∴ 30.next = 20



Removed = 10 Rear = 30
Front = 20

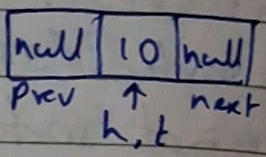
② remove 20 :
front = 30 , Rear = 30



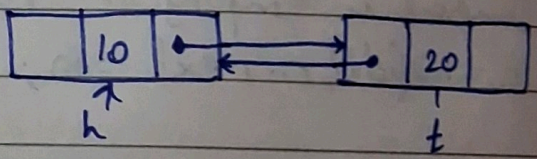
if (front == rear) ∴ rear = null ie empty

⑧ Doubly linked list

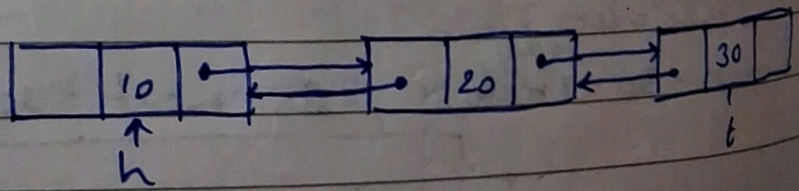
① head = null
tail = null
List.insertAtBeginning (10);
head = tail = newNode;



② List.insertAtEnd (20);
new node = 20
tail.next = newNode;
newNode.prev = tail;
tail = newNode

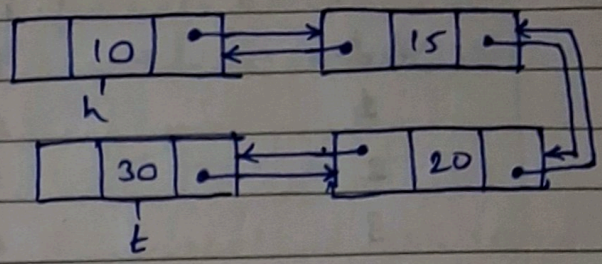


③ List.insertAtEnd (30);
new node = 30
20.next = 30
30.prev = 20
tail = 30



④ List.insertAtPosition (15, 2);
current = 10 , nextNot = 20

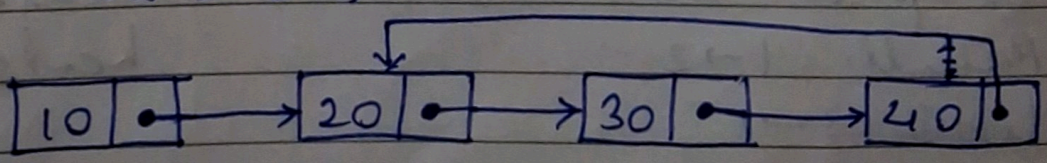
current.next = newNode
newNode.prev = current
newNode.next = nextNode
nextNode.prev = newNode



⑤ List.displayForward();
Node.current = head;
current = 10.
loop: current = 10 → print 10
current = 15 → prints 15
current = 20 → prints 20
current = 30 → print 30
Forward = 10, 15, 20, 30

⑥ List.displayBackward();
Node.current = tail;
current = 30
Here we have prev. move backward
30 → 20 → 15 → 10 → null
Backward: 30, 20, 15, 10

⑦ Home challenge (Cycle Detection)
Pointers:
1. slow = moves 1 node
2. fast = moves 2 nodes



① List is:
The cycle is 20 → 30 → 40 → 20
② Initially: slow = 10
fast = 10

	slow	fast
Start	10	10
1	20	30
2	30	20
3	40	40

at step 3 : slow == fast ie 40 == 40
cycle present : true

③ without cycle: 10 → 20 → 30 → 40 → null
fast eventually reaches null.

Complexity:
n = 10 → around 5 iterations
n = 100 → around 50 iterations
Here, n doubles → work also roughly doubles → O(n)
as slow moves 1 steps ie one loop so O(n)
Fast moves 2 steps → O(2n) → O(n)
so, the complexity is O(n).

★ Merge Sorted list

① List 1: 1 → 3 → 5
List 2: 2 → 4 → 6

② Compare 1 < 2 take 1 if(head1 <= head2) data
Result = 1 head1 = head1.next;
List1 : 3 → 5
List 2: 2 → 4 → 6

③ Compare 3 > 2 take 2 if(head1 >= head2)
Result = 1 → 2 head1 = head2.next
List 1 : 3 → 5
List 2 : 4 → 6

④ Compare 3 < 4 take 3 Result = 1 → 2 → 3
Compare 5 > 4 take 4 Result = 1 → 2 → 3 → 4

Compare 5 < 6 take 5 Result = 1 → 2 → 3 → 4 → 5
Remains only 6 so Result = 1 → 2 → 3 → 4 → 5 → 6

